

Maximum Element in an Array:-

↓ ↓
[100, 200, 300]

(100)

↓
maxi
~~100~~

200

(300)

maxi = ~~5~~ ~~6~~ ~~8~~ 100
↓
Ind

for
[5, 1, 2, 6, 8, 3, 100]

200 > 100 →
300 > 200 →

1 > 5 X

2 > 5 X

6 > 5 ✓

8 > 6 ✓

3 > 8 X

100 > 8 ✓

Minimum

[5, 4, 1, 8, 6, 0, 10]

> <

5 < 4
(4 < 1)

if (arr[i] < min) {
min = arr[i];

4 < 5 ✓

1 < 4 ✓

8 < 1 X

6 < 1 X

0 < 1 ✓

10 < 0 X

min
5
4
1
0

min

4

arr[i]

4

i = 1

int arr = { 5, 4, 1, 8, 6, 0, 10 };

int min = arr[0];

for (int i = 1; i < arr.length; i++) {

if (arr[i] < min) {

min = arr[i];

}
Sort(min);

}

Reverse Print

$[10, 20, 30, 40, 50]$
0 1 2 3 4 ↓

arr[4] arr[3] arr[2] arr[1]

for (int i = arr.length - 1; i >= 0; i--) {
 Print(arr[i]);
}

(arr.length = 5)

$[10, 20, 30, 40, 50, 60]$
 $\uparrow$
 $arr[0]$
 $\uparrow$
 $left$
Swap I $\rightarrow [60, 20, 30, 40, 50, 10]$
 $\uparrow$ $\uparrow$
 $left$ $right$

Pointer
 $\downarrow$
 is a variable that
 point to the given index

$while (l < r) \{$

Swap II $\rightarrow [60, 50, 30, 40, 20, 10]$
 $\uparrow$ $\uparrow$
 $left$ $right$

Swap III $\rightarrow [60, 50, 40, 30, 20, 10]$
 $\uparrow$ $\uparrow$
 $right$ $left$

[7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17]
↓ (left) (right)

Swap I:- [17, 8, 9, 10, 11, 12, 13, 14, 15, 16, 7]
↑ left ↑ right

Swap II:- [17, 16, 9, 10, 11, 12, 13, 14, 15, 8, 7]
↑ left ↑ right

Swap III:- [17, 16, 15, 10, 11, 12, 13, 14, 9, 8, 7]
↑ left ↑ right

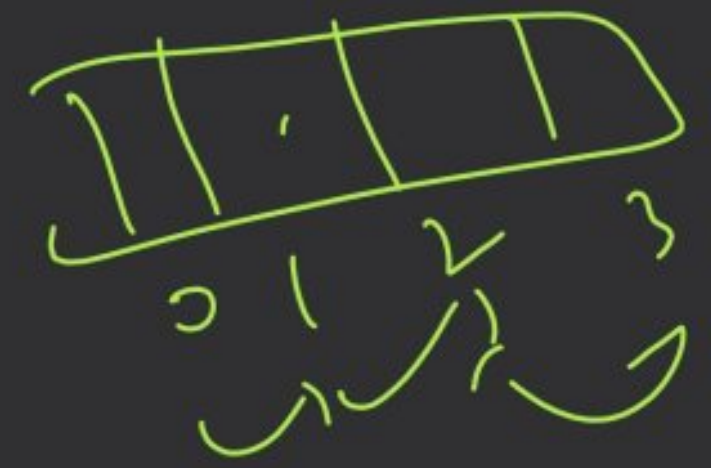
Swap IV

(17, 16)
↓
Swap V:- [17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7]

while (true)
{ L = 0

$[5, 6, 7] \leftarrow [7, 6, 5]$

$[5, 6, 7, 8] \rightarrow [8, 6, 7, 5] \leftarrow [8, 7, 6, 5]$



int arr = { 0; int left = 0; right = arr.length - 1;

while (left < right) {

int temp = arr[left];

arr[left] = arr[right];

arr[right] = temp;

left++, right--;

}

$[5, 6, 7, 8, 9]$
↑ left ↑ right

$[9, 6, 7, 8, 5]$
↑ left ↑ right

[

Reverse part of Array:-

[10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
0 1 2 3 4 5 6 7 8 9 10 11
left right

(indices 2 to 8)

(left = 2
right = 8)

s.length - 1



helcode 344

fill 0
0 1 2 3 4

Char

temp = s[244];
↓
H

0ll, H

Leetcode 1480 :-

$$\underline{\text{nums}[i]} = \text{nums}[i] + \underline{\text{nums}[i-1]}$$

Index

0

1

2 ←

}

$$\text{arr}(0) = 1 \quad \text{arr}(1) = 3$$

Calculation

$$\underline{1} \\ 1 + 2 = \underline{3}$$

$$3 + 3 = \underline{6}$$

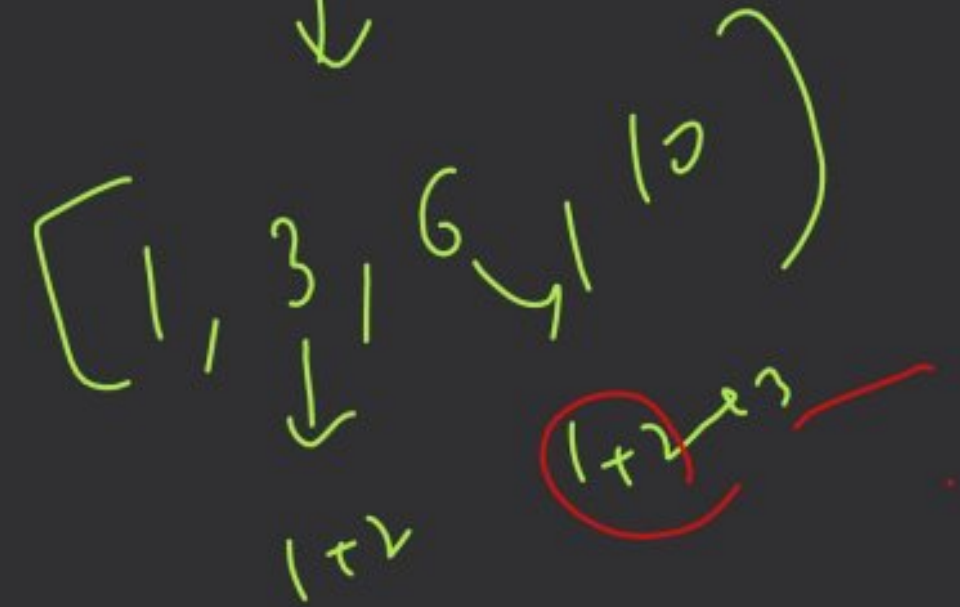
$$6 + 4 = 10$$

Result

$$\underline{1} \\ \underline{3}$$

$$\underline{6}$$

10



(nums)

344. Reverse String

Solved

[Discuss Approach >](#)[Easy](#) [Topics](#) [Companies](#) [Hint](#)

Write a function that reverses a string. The input string is given as an array of characters `s`.

You must do this by modifying the input array **in-place** with $O(1)$ extra memory.

Example 1:

Input: `s = ["h","e","l","l","o"]`

Output: `["o","l","l","e","h"]`

Example 2:

Input: `s = ["H","a","n","n","a","h"]`

Output: `["h","a","n","n","a","H"]`

9.5K | 307 | ☆ | 🔗 | ⓘ

152 Online

https://leetcode.com/subscribe/?ref=lp_pl&source=qd

Code

Java Auto

```
1 class Solution {  
2     public void reverseString(char[] s) {  
3         int left = 0;  
4         int right = s.length - 1;  
5  
6         while(left < right){  
7             char temp = s[left];  
8             s[left] = s[right];  
9             s[right] = temp;  
10  
11             left++;  
12             right--;  
13         }  
14     }  
15 }
```

→ char is used to
characters.

[Help in coding >](#) Saved[Test Result](#) | ☒ Testcase

You must run your code first

Ln 1, Col 1



Search



1295. Find Numbers with Even Number of Digits

Discuss Approach >

Easy Topics Companies Hint

Given an array `nums` of integers, return how many of them contain an even number of digits.

Example 1:

Input: `nums = [12,345,2,6,7896]`

Output: 2

Explanation:

12 contains 2 digits (even number of digits).

345 contains 3 digits (odd number of digits).

2 contains 1 digit (odd number of digits).

6 contains 1 digit (odd number of digits).

7896 contains 4 digits (even number of digits).

Therefore only 12 and 7896 contain an even number of

Code

Java Auto

```
1 class Solution {
2     public int findNumbers(int[] nums) {
3
4     }
5 }
```

→ steps:- Take a number
 step I:- Count digit → digit = 7 8 9 6
 step II:- check for even → Count = 4 X 2

Help in coding > Saved

Test Result | Testcase

You must run your code first

Code

Java Auto

```
1 class Solution {
2     public int findNumbers(int[] nums) {
3         int count = 0; //to count even number of digits
4
5         for(int i = 0; i < nums.length; i++){
6             int digitCnt = 0;
7             int num = nums[i];
8
9             while(num > 0){
10                 num = num / 10;
11                 digitCnt++;
12             }
13
14             if(digitCnt % 2 == 0)
15                 count++;
16         }
17         return count;
18     }
19 }
```

(60, 1233, 782, 2, 862, 73)

Digit cnt

60 → 2

1233 → 4

782 → 3

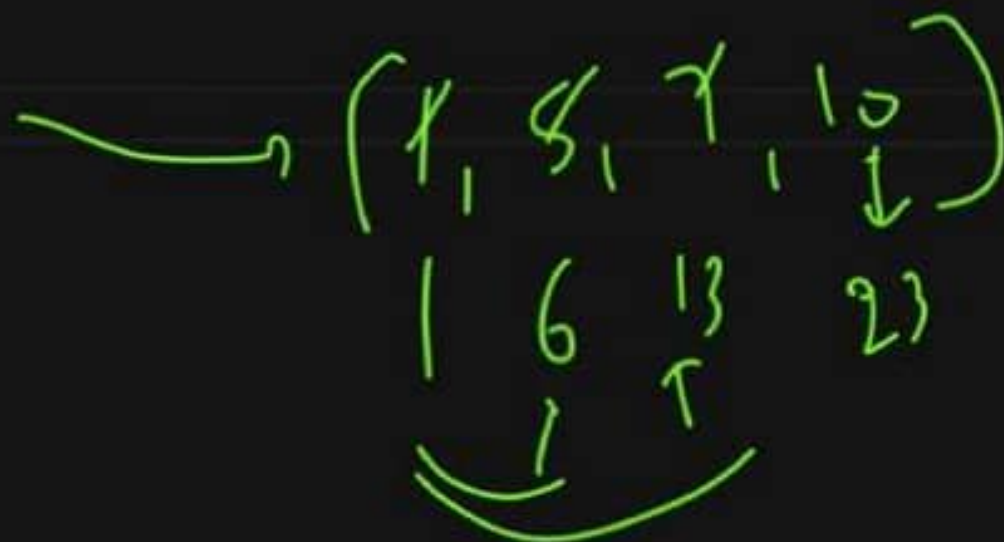
2 → 1

862 → 3

73 → 2

I count = 4


```
1 class Solution {  
2     public int[] runningSum(int[] nums) {  
3         for(int i = 1; i < nums.length; i++) {  
4             nums[i] = nums[i] + nums[i-1];  
5         }  
6     }  
7 }
```



Help in coding >

Saved

Ln 4, Col 43

> Test Result | ☒ Testcase

You must run your code first





Probl...



1:1 Sessions with me



 Code

Java   Auto

```
1 class Solution {
2     public int[] runningSum(int[] nums) {
3         for(int i = 1; i < nums.length; i++) {
4             nums[i] = nums[i] + nums[i-1];
5         }
6         return nums;
7     }
8 }
```



Help in coding >

Saved